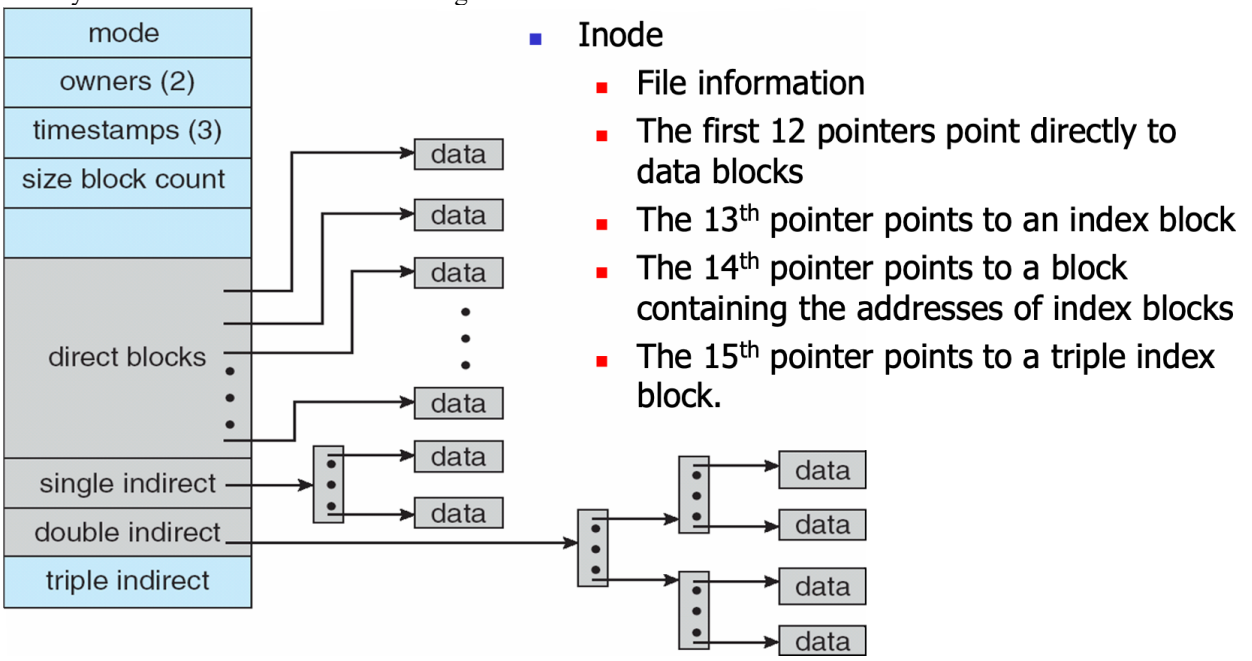
CSS 430 Operating Systems
Program 5: Performance Analysis of Linux File Accesses

1. Purpose
This assignment checks if Linux or Unix multi-level indexed disk block allocation really performs slower as a file grows.

2. Linux Multi-Level Indexed Allocation
As we studied in class, Linux or Unix uses a multi-level indexed allocation combined with the first 12 blocks directly accessed from an inode. See the figure below:



This is true even in our Rocky Linux while its block size is 4KB rather than 512B.

```
[css430@csslab1 ~]$ lsblk -o NAME,PHY-SeC
NAME        PHY-SEC
sda            4096
├─sda1         4096
├─sda2         4096
└─sda3         4096
  ├─rl-root    4096
  ├─rl-swap    4096
  └─rl-home    4096
sr0             512
[css430@csslab1 ~]$ stat -f /
  File: "/"
    ID: fd0000000000 Namelen: 255     Type: xfs
Block size: 4096       Fundamental block size: 4096
Blocks: Total: 10801152   Free: 9048869    Available: 9048869
Inodes: Total: 21635072   Free: 21443027
```

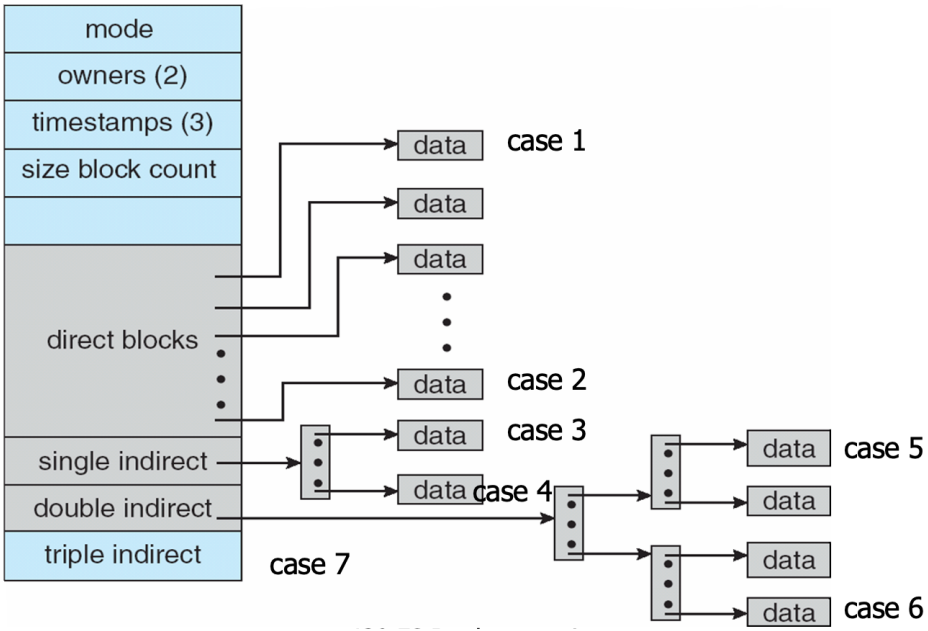
More details are give:
https://www.kernel.org/doc/html/latest/filesystems/ext4/ifork.html

Unlike NTFS, there is no guarantee that consecutive disk blocks are allocated from the free block list to direct pointers 1-12, as well as single indirect, double indirect, and triple indirect blocks. In addition, as a file grows, its block accesses need to go through more indirect blocks, which may thus increase the disk block access latency. This programming project intends to check Linux file access latencies as the file length increases.

Check out ~css430/prog/5/faccesses_template.cpp. This program includes the following functions:

Function name	Descriptions
filewrite( )	1st argument: const char* filename 2nd argument: int nblocks 3rd argument: char* block Opens a file named "filename" for writing. If the file does not exist, open( ) needs to create it.

	Then, this function starts a timer, repeats nblocks times for write block's 4096B, closes the file, and stops the timer. In other words, filewrite( ) creates a file with N blocks. You have to complete this function.
fileread( )	1st argument: const char* filename 2nd argument: int nblocks 3rd argument: char* block Opens a file named "filename" for reading. Then, this function starts a timer, repeatss reading 4096B nblocks time, stop the timer, and closes the file. In other words, fileread( ) entirely reads a file with N blocks. You have to complete this function.
filename( )	1st argument: int i Receives i, generate a file name "f_i.txt", and return it to main( ).
main( )	Initialize a 4096B buffer named block with repetitions of alphabets: a, b, c, …, z. It then: **Case 1:** Creates a file named f_1.txt with 1 block pointed by the first direct pointer. **Case 2:** Creates a file named f_12.txt with 12 blocks pointed by the first all the way to the last direct pointer. **Case 3:** Creates a file named f_N.txt with N blocks pointed by all the direct pointers as well as the first pointer of the single indirect block. **Case 4:** Creates a file named f_N.txt with N blocks pointed by all the direct pointers as well as the first pointer all the way to the last pointer of the single indirect block. **Case 5:** Creates a file named f_N.txt with N blocks pointed by all the direct pointers, all the pointers of the single indirect block as well as the first pointer of the double indirect block. **Case 6:** Creates a file named f_N.txt with N blocks pointed by all the direct pointers, all the pointers of the single indirect block as well as all the way to the last pointer of the double indirect block. **Case 7:** Creates a file named f_N.txt with N blocks pointed by all the direct pointers, all the pointers of the single indirect block, all the pointers of the double indirect block as well as the first pointer of the triple indirect block. After creating/writing these seven files, main( ) reads these files. You need to choose N for test cases 3-7.



According to https://www.kernel.org/doc/html/latest/filesystems/ext4/ifork.html, Rocky Linux, which is the operating system for csslab1-23, allocates 4 bytes for each direct or indirect pointer, so that you can estimate the number of pointers in the single, double, and triple blocks.

3. Statement of Work

Task 1: Copy ~css430/prog/5/faccesses_template.cpp into *faccesses.cpp* and complete its filewrite( ), fileread( ), main( ).

Task 2: Compile and run *faccesses.cpp* as follows:

```
[css430@csslab1 5]$ g++ faccesses.cpp -o faccesses
[css430@csslab1 5]$ ./faccesses
writing to direct blocks ****************************************
nblocks = 1, total time = 6164, time / block = 6164
nblocks = 12, total time = 8426, time / block = 702
writing to 1st indirect blocks ***********************************
nblocks = …
…
```

Take a snapshot of your *faccesses* outputs.

Task 3: Answer the following questions, based on your results (but not in general like chatGPT or Google AI support answers).

Q1. Did the elapsed time per block writing increase as the number of blocks grew?
Q2. Did the elapsed time per block reading increase as the number of blocks grew?
Q3. Did you see any distinct performance increase or decrease across the direct to the single indirect blocks, the single to double indirect blocks, and/or the double to triple indirect blocks?
Q4. Based on Q1-Q3, do you think that Linux randomly allocates free blocks to a file?
Q5. Based on Q1-Q3, besides the factor in Q4, what else do you think affected Linux file accesses?

4. What to Turn in

	Materials	Points
1	Task 1: Your faccesses.cpp a. Code organization: **+2pts** • Well organized: 2pts, • Poor comments or bad organization: 1pt, or • No comments and horrible code: 0pts b. Correctness: **+13pts** • Correctly opened a file for writing: 1pt • Correctly opened a file for reading: 1pt • Correctly started a timer: 1pt • Correctly wrote a give number of blocks: 1pt • Correctly read a given number of blocks: 1pt • The first pointer in the single indexed block: 1pt • The last pointer in the single indexed block: 1pt • The first pointer in the double indexed block: 1pt • The last pointer in the double indexed block: 1pt • The first pointer in the triple indexed block: 1pt	12
2	Task 2: Your execution output: **+3pts** • Correct performance evaluation: 3pts, • Wrong performance evaluation: 1pt, or • No outputs: 0pts	3
3	Task 3: Your answers to the questions: **+5pts** • Q1: 1pt • Q2: 1pt • Q3: 1pt • Q4: 1pt • Q5: 1pt	5

Total 20pts.